

SOLUTIONS TO SOLVE CYCLIC POLICY:

Solution 1: Elevated Privilege Bypass for Cyclic RLS Detection

Simulate SECURITY DEFINER on Cycle Detection

When `fireIRIRules()` detects that the same OID is appearing second time in `activeRIRs` instead of throwing an error, temporarily elevate the session to superuser context — similar to how `SECURITY DEFINER` works — so that the second RLS check is bypassed. The first policy application happens normally, only the recursive second hit is bypassed.

Touch points:

- `fireIRIRules()` in `rewriteHandler.c` — detect second OID hit, trigger elevation
- `get_row_security_policies()` in `rowsecurity.c` — already has superuser bypass logic, will naturally return no policy under elevated context
- Restore original user context immediately after the cyclic relation is processed

Key question to resolve:

- Is temporarily flipping the current user context mid-rewrite safe and reversible without side effects on the rest of the query?

Bottleneck:

It is technically possible but deeply unsafe.

Why technically possible:

PostgreSQL already has `GetUserId()` and `SetUserId()` functions in `miscadmin.h`. We can save current user, flip to superuser context, run RLS check, and restore. This is exactly what `SECURITY DEFINER` does internally.

Why deeply unsafe:

The problem is the rewriter does not execute in isolation. When we flip the user context mid-rewrite:

- The elevated context does not just affect RLS — it affects **every security check happening in parallel** within that same query tree expansion
- Other relations being expanded in the same `fireIRRules()` loop will also see the elevated context and bypass their RLS too — not just the cyclic one
- There is no clean boundary that says "elevate only for this one relation, keep normal for everything else"

The fundamental issue: `SetUserId()` is a **global state change** in PostgreSQL backend. It is not scoped to a single relation or a single RLS check. So even if we save and restore correctly, the window between set and restore touches more than just the cyclic policy.

Short verdict:

Possible to implement in code — but the blast radius of flipping global user context mid-rewrite makes it too risky. It would silently bypass RLS on unintended relations within the same query.

Solution 2: `allow_cycle` Flag as a Controlled Bypass Mechanism

Introduce a New `allow_cycle` Flag on Policy Definition

Add a new flag directly on the policy itself — similar to how `SECURITY DEFINER` is a flag on a function. Only a superuser can set this flag on a policy. When cycle is detected in `fireIRRules()`, instead of erroring, it checks if the second-hit policy has `allow_cycle = true`. If yes, skip the second evaluation and continue. If no, throw error as usual.

How it would look:

```
ALTER POLICY tenant_policy ON customers SET (allow_cycle = true);
```

Touch points:

- `pg_policy` catalog — add new boolean column `polallowcycle`
- `rowsecurity.c` → `get_row_security_policies()` — fetch and pass the flag forward
- `rewriteHandler.c` → `fireIRRules()` — check flag on second OID hit instead of immediately erroring

Key advantages:

- Superuser explicitly grants bypass — not automatic
- Bypass is scoped to **one specific policy** — not global context change
- Existing behavior unchanged for all policies without the flag
- No global state manipulation like `SetUserId()`

Key question to resolve:

- Does skipping the second policy evaluation on a flagged policy still guarantee correct and secure row filtering for the first evaluation?

Question 1: Is the flag feasible for inline policies (no UDF)?

Yes — and actually this flag makes **more sense** for inline policies. Because inline policies are exactly where we cannot use `SECURITY DEFINER` today. UDFs already have their own `SECURITY DEFINER` keyword, so inline policies are the gap this flag fills.

Question 2: Can we reuse `security_definer = true` directly in `CREATE POLICY` instead of a new flag?

Technically yes we can introduce it as:

```
CREATE POLICY tenant_policy ON customers
USING (user_id = current_user_id)
WITH security_definer = true;
```

But there is a **naming problem** — `SECURITY DEFINER` on a function means "run as the function owner." On a policy that concept does not directly translate. It would confuse developers because same keyword means different thing in different context.

Verdict:

Introduce a **new flag with a clear name** scoped specifically to policies:

```
CREATE POLICY tenant_policy ON customers
USING (user_id = current_user_id)
WITH (allow_cycle = true);
```

This is:

- Only applicable on `CREATE POLICY` / `ALTER POLICY`
- Only settable by superuser
- Clearly named — no confusion with function `SECURITY DEFINER`
- Fills the exact gap that inline policies have today

Is this feasible?

1. Catalog perspective Adding a new boolean column `polallowcycle` to `pg_policy` is straightforward. PostgreSQL has done this before when new policy features were added. Requires a catalog version bump but that is standard procedure.

2. Parser perspective `CREATE POLICY` and `ALTER POLICY` grammar is defined in `gram.y`. Adding a new `WITH (allow_cycle = true)` option follows the exact same pattern as existing storage parameters. Low complexity change.

3. Security perspective Safe because:

- Only superuser can set it
- It is explicit — admin consciously decided this policy can be skipped on cycle
- Bypass is scoped to one specific policy — not global
- First policy evaluation still happens normally — no rows leak on first hit

4. Rewriter perspective `fireIRRules()` already has the OID check logic. Adding one catalog lookup to check `polallowcycle` before throwing the error is a minimal change.

5. Backward compatibility perspective Default value of `polallowcycle` is `false` — so all existing behavior is completely unchanged unless admin explicitly opts in.

One genuine concern: Catalog lookup inside `fireIRRules()` adds a small overhead on every RLS cycle detection hit. But since this is an error path that rarely triggers

in healthy schemas, the overhead is negligible.

Conclusion ⇒ We can implement this and test it locally. The only reason I am still not fully confident in this solution is because I am not sure whether it is good from a security point of view. So we can implement it locally and apply multiple queries to test this.

Next task ⇒ Implement this solution locally and test it against multiple queries to validate the security and correctness.

Test	What to verify
Normal cyclic policy without flag	Still throws error — backward compat
Cyclic policy with flag set by superuser	Bypasses second hit, returns correct rows
Cyclic policy with flag, set by non-superuser	Should be rejected at <code>ALTER POLICY</code> time
Three way cycle $A \rightarrow B \rightarrow C \rightarrow A$ with flag	Does it resolve or still loop
Flag set on one policy in cycle but not other	What happens
Multiple policies on same table with cycle	Correct rows returned or not
RLS bypass user querying flagged table	No double bypass
Legitimate non-cyclic policy with flag set	Normal behavior unchanged

Solution 3: Cycle Detection at Policy Creation Time

Reject Cyclic Policies at `CREATE POLICY` Time

When a new policy is being created, before writing it to `pg_policy`, walk the policy expression tree and extract all relation OIDs it references. Then recursively fetch policies of those relations and check if the current relation appears anywhere in the dependency chain. If a cycle is detected, reject the `CREATE POLICY` with a clear error message showing the full cycle path instead of allowing it to be created and failing later at query execution time.

How it would look:

```
CREATE POLICY tenant_policy ON customers
USING (user_id IN (SELECT user_id FROM orders));
-- ERROR: policy would create a cyclic dependency: customers
--> orders --> customers
-- HINT: restructure your policy to avoid referencing tables
that reference back to customers
```

Touch points:

- `rewriteDefine.c` → `CreatePolicy()` — add dependency graph walk before writing to catalog
- `pg_policy` catalog — read existing policies of referenced relations
- Expression tree walker — extract all relation OIDs from policy expression

Key question to resolve:

- How deep should the dependency graph walk go and how to handle policies that reference views which internally reference back?

Discussion:

1. Will this make PostgreSQL more strict?

Yes — this is the biggest tradeoff. There might be legitimate use cases where someone intentionally references another table in a policy that happens to have a back-reference. Today that works until it hits a real cycle at query time. With this solution it gets rejected at creation time even if the actual query execution path would never trigger the cycle.

2. Which part and how will cycle detection work?

Detection happens in `rewriteDefine.c` inside `CreatePolicy()`. The algorithm:

1. Parse the policy USING expression
2. Walk the expression tree → extract all referenced relation OIDs
3. For each referenced relation → fetch its policies from `pg_policy`
4. From those policies → extract their referenced relation OIDs
5. Repeat recursively
6. If current relation OID appears anywhere in the graph → CYCLE → reject

The expression tree walker already exists in PostgreSQL — `pull_vars_of_level()` and `relation_walker` patterns are used elsewhere. So this is not building from scratch.

Pros and cons:

Pros	Cons
Problem prevented at source	Makes policy creation stricter
Clear error message with full cycle path	May reject valid schemas that never actually cycle at runtime
No runtime overhead in rewriter	Detection at creation time is complex — views inside policies add layers
No security tradeoffs	Does not fix existing databases with cyclic policies already present
No new flags or catalog changes needed	Graph walk can be expensive for complex policy chains

3. Can we introduce `check_cycle = true` flag instead of always checking?

Yes and this is actually the **better design**:

```
-- default behavior unchanged
CREATE POLICY tenant_policy ON customers
USING (user_id = current_user_id);

-- opt in to cycle check
CREATE POLICY tenant_policy ON customers
USING (user_id = current_user_id)
WITH (check_cycle = true);
```

This way:

- Existing behavior is completely unchanged
- Users who want protection opt in explicitly
- No risk of rejecting legitimate schemas
- PostgreSQL philosophy of explicit over implicit is respected

Deep Analysis between solution 2 and 3 because they look feasible.

Security POV

- Creation check → **safer** — cycle never enters the system
- Flag solution → security depends on admin correctly setting it — human error possible

Performance POV

- Creation check → one time cost at DDL, zero runtime overhead
- Flag solution → catalog lookup on every cycle hit during rewriting

Practicality POV

- Creation check → does not help existing databases with cycles already present
- Flag solution → works for both new and existing schemas

PostgreSQL Philosophy POV

- Creation check → **better fit** — fail fast, fail early, fail clearly
- Flag solution → acceptable but relies on admin intervention after the fact

Complexity POV

- Creation check → harder to implement — expression tree walker, view expansion, recursive graph walk
- Flag solution → simpler — catalog column + one condition in `fireIRRules()`

```
testing=> CREATE VIEW view_a AS SELECT 1 AS id;
CREATE VIEW
testing=> CREATE VIEW view_b AS SELECT 1 AS id;
CREATE VIEW
testing=> CREATE OR REPLACE VIEW view_a AS SELECT * FROM view_b;
CREATE VIEW
testing=> CREATE OR REPLACE VIEW view_b AS SELECT * FROM view
```

```

_a;
CREATE VIEW
testing=> SELECT * FROM view_a;
NOTICE:  === REWRITER INPUT ===
NOTICE:  Command type: 1
NOTICE:  Query string: original
NOTICE:  Input Query tree: {QUERY :commandType 1 :querySource
0 :canSetTag true :utilityStmt <> :resultRelation 0 :hasAggs
false :hasWindowFuncs false :hasTargetSRFs false :hasSubLinks
false :hasDistinctOn false :hasRecursive false :hasModifyingC
TE false :hasForUpdate false :hasRowSecurity false :hasGroupR
TE false :isReturn false :cteList <> :rtable ({RANGETBLENTY
:alias <> :eref {ALIAS :aliasname view_a :colnames ("id")} :r
tekind 0 :relid 16416 :inh true :relkind v :rellockmode 1 :pe
rminfoindex 1 :tablesample <> :lateral false :inFromCl true :
securityQuals <>}) :rteperminfos ({RTEPERMISSIONINFO :relid 1
6416 :inh true :requiredPerms 2 :checkAsUser 0 :selectedCols
(b 8) :insertedCols (b) :updatedCols (b)}) :jointree {FROMEXP
R :fromlist ({RANGETBLREF :rtindex 1}) :quals <> :mergeActio
nList <> :mergeTargetRelation 0 :mergeJoinCondition <> :targe
tList ({TARGETENTRY :expr {VAR :varno 1 :varattno 1 :vartype
23 :vartypmod -1 :varcollid 0 :varnullingrels (b) :varlevelsu
p 0 :varreturningtype 0 :varnosyn 1 :varattnosyn 1 :location
-1} :resno 1 :resname id :ressortgroupref 0 :resorigtbl 16416
:resorigcol 1 :resjunk false}) :override 0 :onConflict <> :re
turningOldAlias <> :returningNewAlias <> :returningList <> :g
roupClause <> :groupDistinct false :groupingSets <> :havingQu
al <> :windowClause <> :distinctClause <> :sortClause <> :lim
itOffset <> :limitCount <> :limitOption 0 :rowMarks <> :setOp
erations <> :constraintDeps <> :withCheckOptions <> :stmt_loc
ation -1 :stmt_len -1}
NOTICE:  >>> fireRIRrules called, activeRIRs length = 0
NOTICE:  >>> fireRIRrules called, activeRIRs length = 1
NOTICE:  >>> fireRIRrules called, activeRIRs length = 2

```

```
ERROR:  infinite recursion detected in rules for relation "view_a"
```

```
CREATE VIEW view_a AS SELECT 1 AS id;
CREATE VIEW view_b AS SELECT 1 AS id;

CREATE OR REPLACE VIEW view_a AS SELECT * FROM view_b;
CREATE OR REPLACE VIEW view_b AS SELECT * FROM view_a;

SELECT * FROM view_a;
-- ERROR: infinite recursion detected in rules for relation
"view_a"
```

	Solution 2	Solution 3
<code>allow_cycle</code> flag	For existing schemas	—
Creation time detection	—	For new schemas

And they are complementary not competing.

For your RLS problem, DFS-based cycle detection is what we need for Solution 3 at creation time. But no algorithm can decide **what rows to return** when a cycle exists — that is why our Solution 2 flag exists — to let the human make that semantic decision.